

AI 调试对决实验报告

团队： 海底小纵队 | 日期： 2026-05-19 | AI 工具： Claude Code

一、实验设计

本实验选择 3 个在系统集成联调中发现的真实 Bug，逐一进行纯人工调试与 AI 辅助调试的对比，记录定位耗时、定位准确度和修复方案质量，重点分析人工调试的核心价值与 AI 辅助的互补作用。

二、Bug 对比记录

Bug #1：需求列表分页"下一页"按钮异常禁用

Bug 描述： 需求列表在存在 CANCELED（已取消）状态需求时，分页"下一页"按钮被异常禁用，即使数据库中仍有更多数据。

复现步骤：

- 发布若干需求，将其中 1-2 个取消
- 在需求大厅浏览（需求总数 > pageSize 时）
- 观察分页组件的"下一页"按钮状态

维度	纯人工调试	AI 辅助调试
问题描述	浏览器中翻页被禁用，后端 API 返回正常 (人工先通过业务经验锁定"前端数据处理逻辑"方向)	提供报错信息、RequirementListView.vue 和 listRequirements 的 API 封装 (依赖人工提供精准的上下文文件)
定位耗时	~15 分钟（人工逐步排查：先通过网络请求验证后端数据有效性，再结合业务场景分析前端过滤逻辑对分页的影响，过程中积累了同类场景的调试经验）	~1 分钟（直接定位到 RequirementListView.vue:112，但依赖人工提前筛选的核心文件）
定位准确度	人工通过"后端数据正常→前端行为异常"的逻辑推导，锁定 0 赋值问题，过程中理解了"前端过滤后数组长度≠总数据量"的业务逻辑本质	准确定位到 total.value = requirements.value.length 应为 data.total，但仅停留在代码语法层面，未解释业务逻辑层面的根因
修复方案	人工将 requirements.value.length 改为 data.total，同时考虑到该修改对分页组件整体逻辑的影响，预判了修改后可能的边界场景（如无数据、单页数据等）	将第 112 行和第 136 行两处相同问题一并修复，但未评估修改对分页组件其他功能的潜在影响
方案质量	人工方案基于对业务场景的完整理解，兼顾代码修改与业务逻辑的适配，可应对边界情况	两处一次性修复无遗漏，但仅聚焦代码本身，未考虑业务边界场景
最终修复	以人工方案为核心，结合 AI 提示的两处修改点，完成修复 (人工主导确认修改范围和业务合理性)	-

AI 与人工表现分析：

AI 在代码文本匹配和快速定位语法问题上效率突出，但依赖人工提供精准的上下文文件和前期的问题方向判断；人工调试虽耗时更长，但能从业务逻辑层面理解 Bug 产生的本质，推导过程中形成的"前端过滤逻辑影响分页计算"的经验，可迁移到同类分页问题的调试中。人工的核心价值在于：判断"后端数据正常→前端逻辑异常"的方向，以及评估代码修改对业务场景的整体影响。

Bug #2： LOST_FOUND 筛选类型无后端对应项

Bug 描述： 需求大厅筛选下拉框包含"失物招领"选项（value = LOST_FOUND），但后端 RequirementType 枚举中无此值。用户选择该筛选项后永远返回 0 条结果。

复现步骤：

- 1. 打开需求大厅
- 2. 类型筛选选择"失物招领"
- 3. 始终显示"暂无符合条件的需求"

维度	纯人工调试	AI 辅助调试
问题描述	前端筛选"失物招领"无结果 （人工先根据"筛选无结果"的业务现象， 锁定"前后端枚举一致性"方向）	将 RequirementListView.vue 筛选下拉和 RequirementType.java 枚举提供给 AI（人工筛选核心文件， 排除无关代码干扰）
定位耗时	~10 分钟（人工先通过 Network 面板确认请求参数， 再结合"枚举值需前后端一致"的业务规则， 追踪到后端枚举定义，过程中梳理了前后端枚举同步的规范）	~1 分钟（跨文件搜索对比枚举值， 但依赖人工提供的前后端核心文件）
定位准确度	人工通过"请求参数正确→后端无对应枚举"的逻辑， 定位问题同时， 理解了"前端枚举需与后端保持一致"的业务规范本质	准确定位问题，并附带发现前端 typeLabel 函数中 3 个枚举名不匹配，但未区分"无效枚举 （LOST_FOUND）"和"命名不匹配"的不同业务性质
修复方案	人工评估两种方案：后端添加枚举会导致数据库数据不兼容、 与设计文档冲突，因此优先选择前端删除无效选项 （基于业务规范和历史数据的综合判断）	推荐删除前端 LOST_FOUND，同时补充缺失的其他有效类型， 但未解释为何不选择后端修改的核心原因 （未结合数据库和设计文档等业务背景）
方案质量	人工方案结合数据库现状、P3 设计文档等业务背景， 规避了枚举修改可能带来的兼容性问题，符合整体业务规范	方案覆盖代码层面的一致性，但未考虑数据库兼容性、 设计文档对齐等业务维度
最终修复	人工主导确认"前端同步后端枚举"的核心方向，结合 AI 提示的缺失类型，补充 STUDY_HELP/MATERIAL_SHARE 等（人工审核补充类型的业务合理性）	-

AI 与人工表现分析：

AI 能快速完成跨文件的文本对比，但无法理解"后端添加枚举会导致数据库数据不兼容"这类业务背景；人工虽需逐个排查，但能结合数据库现状、设计文档等非代码层面的业务信息，做出更符合整体业务逻辑的决策。人工的核心价值在于：判断修复方案的业务合理性，规避技术修改带来的业务风险，同时梳理出"前后端枚举同步需先核对设计文档和历史数据"的规范。

Bug #3：typeLabel 函数枚举名称与后端不一致

Bug 描述：前端 RequirementListView.vue 和 RequirementDetailView.vue 中的 typeLabel() 函数使用 STUDY、MATERIAL、QA 等枚举名，但后端 RequirementType.java 实际值为 STUDY_HELP、MATERIAL_SHARE、Q_AND_A。导致需求类型在前端界面显示为原始枚举值（如 STUDY_HELP）而非中文标签。

复现步骤：

- 1. 发布一个类型为 STUDY_HELP 的需求
- 2. 在需求列表中查看该需求的类型显示
- 3. 观察显示的是 STUDY_HELP（英文原始值）而非"学习互助"

维度	纯人工调试	AI 辅助调试
问题描述	需求列表中某些需求的分类显示英文枚举名而非中文 （人工先根据"前端显示异常"的业务现象， 锁定"枚举映射关系"方向）	将 RequirementListView.vue typeLabel、RequirementDetailView.vue typeLabel、RequirementType.java 三个文件提供给 AI （人工精准筛选核心文件）
定位耗时	~12 分钟（人工在两个 Vue 文件和 Java 枚举间对照， 过程中理解了"枚举名映射错误导致前端标签显示异常"的业务影响， 同时记忆了后端枚举的命名规则）	~1 分钟（跨文件全文对比列出不匹配项， 但依赖人工提供的核心文件）
定位准确度	人工逐行对比时，能区分"完全不存在的枚举 （LOST_FOUND）"和"命名不匹配	精确列出所有不匹配项，但未区分问题性质， 仅作为文本差异呈现

维度	纯人工调试	AI 辅助调试
	(STUDY→STUDY_HELP) "的不同问题性质，理解其对用户体验的不同影响	
修复方案	人工逐个修改不匹配的 key，同时考虑到用户体验：修改后验证中文标签显示效果，确保符合用户对"学习互助""资料分享"等标签的认知习惯	一次性修正两个 typeLabel 函数中的所有 key，同步更新筛选下拉，但未验证修改后的前端显示效果是否符合用户体验预期
方案质量	人工方案兼顾代码修改与用户体验，确保中文标签符合用户认知，同时检查修改后其他枚举类型的显示是否正常	代码层面一次性修正所有不匹配项，但未评估修改对用户体验的影响
最终修复	人工主导确认修改范围，结合 AI 提示的不匹配项，完成修改后验证前端显示效果（人工保障最终体验符合业务预期）	-

AI 与人工表现分析：

AI 能高效完成多文件的文本对比，但仅关注代码语法层面的一致性；人工虽易遗漏个别不匹配项，但能从用户体验角度评估修改效果，理解"枚举名映射错误导致用户看到英文值"的业务影响，同时记忆后端枚举命名规则，为后续开发规避同类问题。人工的核心价值在于：将代码修改与用户体验结合，验证修改后的实际效果，形成可复用的枚举命名规则记忆。

三、实验数据汇总

Bug #	Bug 描述	纯人工定位耗时 (估)	AI 辅助定位耗时 (实)	AI 定位是否准确	AI 修复方案是否可用	最终方案来源
1	分页 total 被前端覆盖	~15 min	< 1 min	准确 (依赖人工提供上下文)	可用 (需人工评估业务合理性)	人工主导 + AI 辅助
2	LOST_FOUND 无后端对应	~10 min	< 1 min	准确 (依赖人工筛选文件)	可用 (需人工判断业务兼容性)	人工主导 + AI 辅助
3	typeLabel 枚举名称不匹配	~12 min	< 1 min	准确 (依赖人工提供核心文件)	可用 (需人工验证用户体验)	人工主导 + AI 辅助

四、分析问题

Q1: AI 在哪类 Bug 上表现较好？

AI 擅长**代码静态文本分析类 Bug**：

- 快速定位语法层面的代码错误（如变量赋值错误、文本不匹配）；
- 跨文件文本对比（如枚举值、函数名一致性）；
- 批量识别重复代码中的同类问题。

但以上表现均依赖人工提前筛选精准的上下文文件、明确调试方向，否则 AI 易陷入无关代码的分析，降低效率。

Q2: 人工在哪类 Bug 上不可替代？

人工在**需结合业务逻辑/场景的 Bug** 上核心价值突出：

- 需理解业务规则的 Bug（如分页逻辑与前端过滤的业务关联、枚举修改与数据库兼容性）；
- 需评估修改风险的 Bug（如后端枚举新增可能导致的历史数据不兼容）；
- 需兼顾用户体验的 Bug（如枚举标签显示是否符合用户认知）；
- 需沉淀经验/规范的场景（如调试中形成的"前后端枚举同步规范""分页逻辑调试思路"）。

Q3: AI 是否出现只修表象、不找根因的情况？

本次实验中，AI 虽能定位代码层面的根因，但未解释**业务逻辑层面的根因**：

- Bug #1: AI 未解释"前端过滤后数组长度≠总数据量"的业务逻辑根因；
 - Bug #2: AI 未说明"为何优先删除前端无效枚举而非新增后端枚举"的业务背景；
 - Bug #3: AI 未分析"枚举名不匹配导致用户体验下降"的业务影响。
- 而人工调试过程中，核心是理解业务层面的根因，代码修改仅为解决问题的手段。

Q4: 人工提供哪些上下文后，AI 的调试建议明显变好？

AI 的效果高度依赖人工的"前置判断"：

- 人工基于业务经验锁定调试方向（如"前后端枚举一致性"），减少 AI 的分析范围；
- 人工筛选核心文件（排除无关代码），避免 AI 无效分析；
- 人工补充业务背景（如数据库现状、设计文档要求），让 AI 方案更贴合实际。

五、结论

本次实验中，AI 辅助调试在**代码层面的定位效率**上表现突出（平均 < 1 分钟/个，较人工提速 10-15 倍），但全程依赖人工的前置判断和上下文筛选；人工调试虽耗时更长，但在**业务逻辑理解、风险评估、用户体验保障、经验沉淀**上不可替代，最终所有修复方案均以人工主导确认方向、评估合理性，结合 AI 的代码层面建议完成。

核心结论：

1. AI 是高效的"代码工具"，擅长快速定位语法级问题、批量处理文本对比，但无法理解业务逻辑、评估修改风险，需人工把控方向和质量；
2. 人工是调试的"核心决策者"，负责判断问题方向、评估方案的业务合理性、保障最终效果符合业务和用户预期，同时沉淀的调试经验可迁移到同类场景；
3. 最优调试模式为"人工主导 + AI 辅助"：人工定方向、控风险、验效果，AI 提效率、批量处理代码层面的重复工作。

后续建议：

- 利用 AI 处理代码静态分析类工作（如枚举对比、重复代码检查），节省人工时间；
- 人工聚焦业务逻辑分析、方案风险评估、经验沉淀，将精力投入到 AI 无法覆盖的维度；
- 建立"人工提供上下文→AI 初步分析→人工审核决策"的调试流程，兼顾效率与业务合理性。